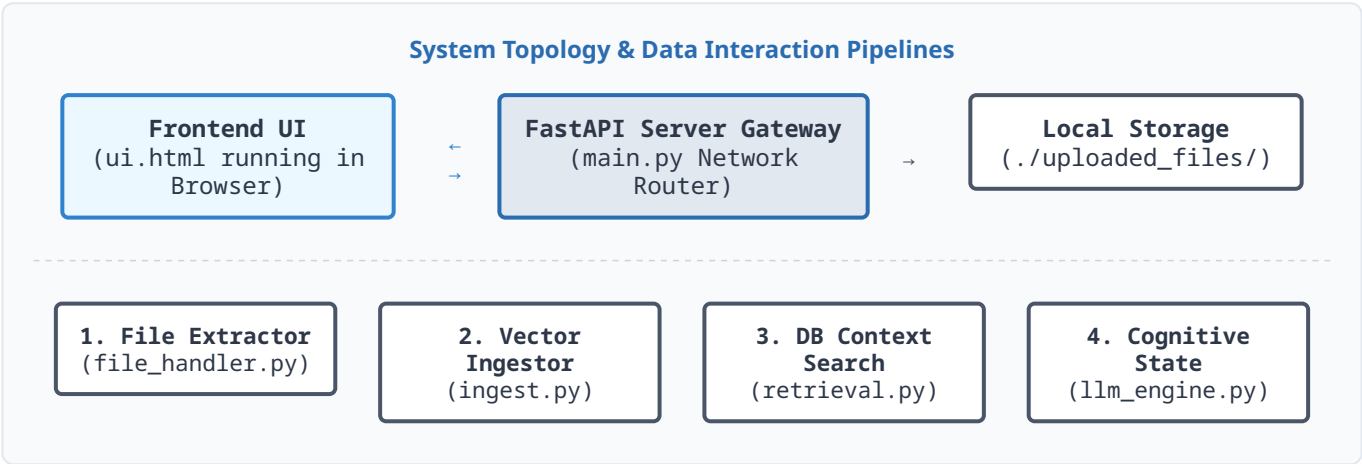


FastAPI Server: Line-by-Line Architectural Masterclass

Welcome to Backend Engineering! This comprehensive documentation provides an absolute line-by-line breakdown of your system's master control file: `main.py`. As a first-year student who understands pipelines and system architectures, you are already thinking like a senior engineer. This guide bridges your architectural clarity directly into the mechanics of the Python code.

1. The Modular Pipeline Architecture

Before examining the specific lines of code, let us map out the high-level infrastructure of your **Retrieval-Augmented Generation (RAG)** application. Your server functions as a centralized router that bridges internet web browsers, your local file system, an vector storage system (Qdrant), and a local large language model (Ollama).



2. Line-by-Line Code Breakdown

Lines 1–4: Custom Module Integrations

```
from ingest import ingest_knowledge
from retrieval import retrieve
from llm_engine import chat as llm_chat
from file_handler import extract_text, SUPPORTED_TYPES
```

These lines load your custom modules into memory. By dividing your system into independent files, you isolate structural faults. If your PDF parser breaks, your LLM engine remains completely operational. Notice the `as llm_chat` statement on Line 3; this creates a localized namespace alias to prevent name collisions with other functions named `chat` across the program.

Lines 5–7: FastAPI Web Core Dependencies

```
from fastapi import FastAPI, UploadFile, File, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import HTMLResponse
```

This imports the network routing building blocks. `FastAPI` acts as the listening daemon for network requests, while `UploadFile` and `File` create specialized buffers that catch binary uploads streaming over HTTP without memory leaks. `HTTPException` is a control-flow shortcut used to interrupt a pipeline if an asset validation fails, returning errors like `400 Bad Request` straight back to the client.

Lines 8–10: Data Schema Enforcers & System Core Utilities

```
from pydantic import BaseModel
import os, json, shutil, datetime
```

`BaseModel` from the Pydantic library enforces strict typing runtime validation for data schemas entering your system. `os` interfaces directly with your system kernel to check or make file folders. `shutil` acts as an optimized storage system tool belt to copy raw binary blocks into storage, and `json` handles structural data translations.

Lines 13–15: Designing the Network Contract

```
class ChatRequest(BaseModel):
    text: str
    history: list = []
```

This explicitly defines your system's network payload schema constraint. It states that whenever a browser sends data to your `/chat` endpoint, it **must** be a JSON packet containing a `text` variable holding a string, alongside an optional array called `history`. If a user tries to send a malicious or broken payload layout, Pydantic intercepts it and kills the connection before it can crash your code.

Lines 17–25: Initializing Server Context

```
app = FastAPI(
    title="Multimodal Cognitive Assistant",
    description=...
),
)
```

This block constructs your server's core execution context. Behind the scenes, instantiation triggers an automated OpenAPI layout pipeline, turning this description parameter directly into interactive live documentation web screens (accessible at `/docs`).

Lines 26–27: Hard Drive State Syncing

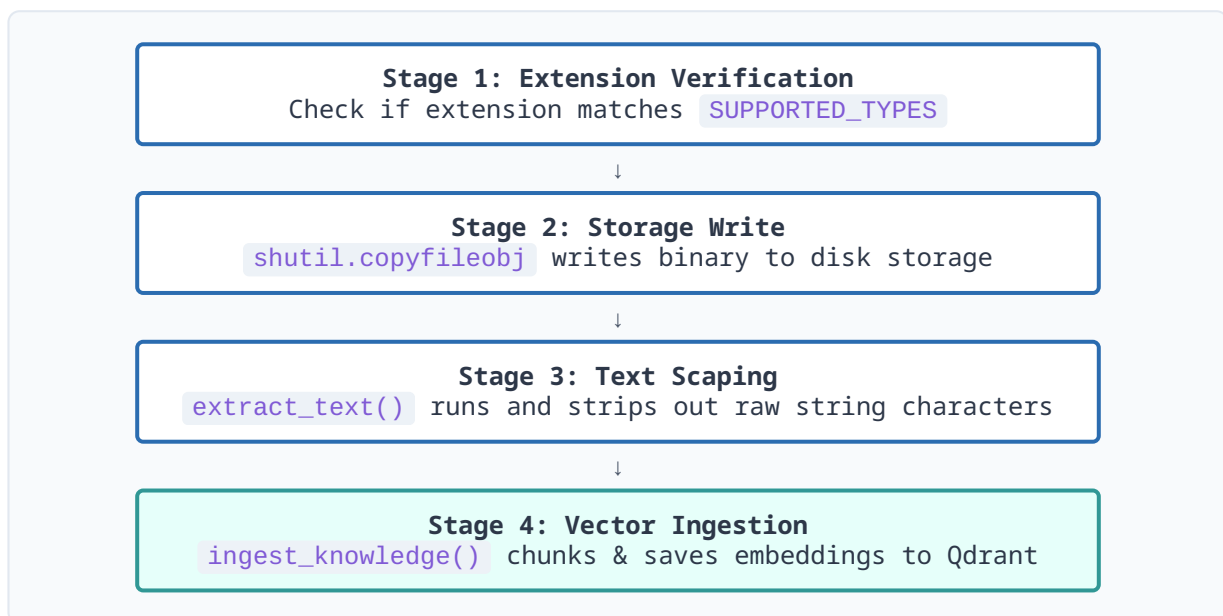
```
UPLOAD_DIR = './uploaded_files'
os.makedirs(UPLOAD_DIR, exist_ok=True)
```

This sets the persistent workspace path on your local hard drive. The parameter `exist_ok=True` is a crucial guardrail: it instructs the kernel that if the folder `./uploaded_files` already exists from a prior runtime loop, don't crash with a directory error; simply bypass and continue.

3. Endpoint Pipeline Flowcharts

The Document Upload Pipeline (`@app.post('/upload')`)

When a user uploads a document through the frontend user interface, the system executes a four-stage translation process to convert a human-readable file into vector coordinates within Qdrant.



```
@app.post('/upload')
async def upload_file(file: UploadFile = File(...)):
    ext = os.path.splitext(file.filename)[1].lower()
    if ext not in SUPPORTED_TYPES:
        raise HTTPException(status_code=400, detail='Unsupported file type')
```

Line 34 splits the file name right at the punctuation barrier to extract the extension (e.g. `.pdf`) and normalizes it to lowercase. Line 35 matches it against your allowed filter list. If it isn't supported, an `HTTPException` is raised immediately, short-circuiting the request and protecting your system from corrupted inputs.

```
dest = os.path.join(UPLOAD_DIR, file.filename)
with style open(dest, 'wb') as f:
    shutil.copyfileobj(file.file, f)
```

Line 38 stitches the folder directory path and the raw file string name together into an explicit hard drive locator path. Line 39 opens that location in **Write Binary ('wb') mode**, which tells your computer to prepare to write raw data packets. `shutil.copyfileobj` streams the binary file across network buffers directly onto your computer disk safely without overloading your RAM.

```
text = extract_text(dest)
txt_path = dest + '_extracted.txt'
with open(txt_path, 'w', encoding='utf-8') as f:
    f.write(text)
```

Once saved, the path is passed to your custom `extract_text` function. This processes the document, strips away structural layout markers, and returns a single clean python string variable. It saves this clean text into an intermediate log file with the `_extracted.txt` suffix for debugging traceability.

```
ingest_knowledge(txt_path)
ingested_files.append({'filename': file.filename, 'path': dest})
return {'status': 'success', 'word_count': len(text.split())}
```

Finally, your system routes the plain text file path into your vector processor (`ingest_knowledge`). This splits the file into small chunks, generates a 384-dimensional mathematical embedding space array for each paragraph, and indexes them permanently into your Qdrant database. It returns a success tracking status back to your browser UI along with a dynamic word count metrics audit.

The RAG Cognitive Loop Chat Endpoint (`@app.post('/chat')`)

```
@app.post('/chat')
async def chat_endpoint(req: ChatRequest):
    context = retrieve(req.text)
    return llm_chat(req.text, context=context, history=req.history)
```

This route orchestrates the entire intelligence loop of your assistant whenever a user posts a conversational question:

Pipeline Step	Underlying Mechanism	Data Transformations
1. User Query Capture	Pydantic validates input from <code>req.text</code> against data models.	JSON Text Object → Python String Variable
2. Qdrant Context Retrieval	Calls <code>retrieve()</code> . The user's query is vectorized. A Cosine Similarity scan pulls the top 5 closest conceptual document matches from memory.	String Prompt → Vector Coordinates → Related Context Text Blocks
3. Cognitive LLM Evaluation	Stitches retrieved context blocks into the prompt templates. Passes the data to Ollama to trace Intent Profiles and mutate the <code>TrustStateMachine</code> state.	Context + Question → Trust Analysis Matrix → Structural Cognitive JSON

4. Server Execution Framework

```
if __name__ == '__main__':
    import uvicorn
    uvicorn.run('main:app', host='0.0.0.0', port=8000, reload=True)
```

Line 57 checks if this file is the primary ignition process running directly inside the terminal console. If true, it imports **Uvicorn**, an asynchronous server engine designed to handle thousands of concurrent requests. The parameter `host='0.0.0.0'` tells the computer to listen on all local network interfaces—allowing you to connect your phone or other computers to your assistant via your local Wi-Fi IP address. Setting `reload=True` sets up a file system watcher: if you save changes to your code, it reboots the server engine automatically, giving you an instantaneous feedback loop as you build your AI application.